Open in Colab

Integer Arithmetic

For addition and subtraction use usual `+` and `-` operators.

In [42]:
```
12 + 3
```

Out[42]: 15

In [43]:
```
15 - 12
```

Out[43]: 3

For multiplication use: `*`

e.g. $12535514 \times 923136652544$

In [44]:
```
12535514 * 923136652544
```

Out[44]: 11571992431878447616

For exponentation use: `**`

e.g. 2^{1024}

In [45]:
```
2**1024
```

Out[45]:
```
179769313486231590772930519078902473361797697894230657273430081157732675805500963132708477322407536021120113879871393357658789768814416622492847430639474124377767893424865485276302219601246094119453082952085005768838150682342462881473913110540827237163350510684586298239947245938479716304835356329624224137216
```

You can also use `pow()` function for exponentation.

In [46]:
```
pow(2, 1024)
```

Out[46]:
```
179769313486231590772930519078902473361797697894230657273430081157732675805500963132708477322407536021120113879871393357658789768814416622492847430639474124377767893424865485276302219601246094119453082952085005768838150682342462881473913110540827237163350510684586298239947245938479716304835356329624224137216
```

Info: Python can hold arbitrary big integers as you can see in the previous result.

To get the absolute value of a number use `abs()` function.

In [47]:
```
abs(-42)
```

Out[47]: 42

To get the maximum of some numbers use `max()` function.

In [48]:
```
max(1, 5, 3, 9, 2)
```

Out[48]: 9

To get the minimum of some numbers use `min()` function.

In [49]:
```
min(1, 5, 3, 9, 2)
```

Out[49]:
```
1
```

Modular Arithmetic

Modular arithmetic is done in Python with builtin `divmod()` function.

In [50]:
```
# Get the division and modulus of 17 divided by 5
divmod(17, 5)
```

Out[50]:
```
(3, 2)
```

If you only need the floor division then:

In [51]:
```
# the division of 17 divided by 5
17 // 5
```

Out[51]:
```
3
```

If you only need the remainder then:

In [52]:
```
# the remainder of 17 divided by 5
17 % 5
```

Out[52]:
```
2
```

Since Python can hold really big numbers, when writing integers, you can use `_` inside integer to put a space for better readablity. Python will ignore that `_` characters.

In [53]:
```
923_136_652_544_123_9774
```

Out[53]:
```
9231366525441239774
```

Operator Precedence

Operator preceedence in Python is like in mathematics.

Operator	Description
`**`	Exponentiation
`+` `-`	Unary plus, unary minus
`*` `//` `%` `/`	Multiplication, floor division, and modulus, division
`+` `-`	Addition and subtraction

For example:

In [54]:
```
-12**3+5/7
```

Out[54]:
```
-1727.2857142857142
```

is equivalent to:

In [55]:
```
(-(12**3))+(5/7)
```

Out[55]:
```
-1727.2857142857142
```

Representing Characters With Integers

Any character in any alphabet can be represented by an integer. For this we need a one-to-one mapping from the alphabet to the integers called *character set*.

- The simplest character set is ASCII which maps the English alphabet to integers between 0 and 127.
- Unicode is the most general character set in the world. It maps all human alphabets including ancient ones like hieroglyphics and cuneiform to numbers between 0 and 1112064.
- For backward compatibility integer representatin of ASCII characters are same in unicode too.

Characters are represented by writing them between `'` or `''` in Python.

In [69]:
```
# To get the integer representation of character 'A'
ord("A")
```

Out[69]: 65

In [70]:
```
# To get the character representation of integer 65
chr(65)
```

Out[70]: 'A'

You can do integer arithmetic on characters. For example:

In [71]:
```
# To convert uppercase 'A' to lowercase 'a'
chr(ord("A") + 32)
```

Out[71]: 'a'

Warning: This may not work for characters other than ASCII characters.

In [72]:
```
chr(ord("İ") + 32)
```

Out[72]: 'Ő'